

Hypervisor & OS-Image Infrastructure

on Oracle Cloud Infrastructure (OCI)

The project explained end to end, in simple terms

The project (as written on the resume): Owned hypervisor and OS-image infrastructure on Oracle Cloud Infrastructure end to end, from base-image creation through validation, publishing, staged rollout, and production troubleshooting across multiple environments and compute configurations.

What is inside this document

- Section 1 - The simple summary (the big picture)
- Section 2 - Key words explained in plain English
- Section 3 - Architecture diagram
- Section 4 - The architecture diagram explained, piece by piece
- Section 5 - Every component explained in detail
- Section 6 - The complete workflow, step by step
- Section 7 - Multiple environments and compute configurations
- Section 8 - Production troubleshooting playbook
- Section 9 - Safety nets, success measures, and a 60-second explanation

Note: The resume line describes the work at a high level. This document explains how such a system is typically designed and run on OCI. Exact tool names (for example Jenkins vs. GitHub Actions, or Packer vs. an in-house builder) can differ from team to team; the ideas and the flow stay the same.

1. The Big Picture (Simple Summary)

1.1 The one-line version

I was responsible for the entire "factory" that creates, tests, releases, and supports the ready-made operating-system templates (images) and the virtualization layer (hypervisor) that Oracle Cloud servers run on - across test, staging, and production, and across many kinds of machines.

1.2 Explained with an everyday example

Think of a big apartment company that owns many large buildings.

- **The building** is a physical server (a real computer in Oracle's data center).
- **The building manager** is the **hypervisor**. It splits one big building into many separate apartments, gives each apartment its own share of water and electricity (CPU and memory), and makes sure no tenant can walk into another tenant's apartment.
- **Each apartment** is a **virtual machine (VM)** - a "computer inside a computer" that a customer rents.
- **The move-in kit** is the **OS image**. Instead of every tenant building their own furniture, the company prepares one perfect, pre-furnished template (the operating system, drivers, security settings, and tools). Every new apartment is copied from this template, so it is ready in minutes.

This project owned the whole life of those move-in kits and the building-manager software: **designing** the kit, **building** it, **inspecting** it in every kind of building, **publishing** it to the catalog, **rolling it out** slowly and safely, and **fixing problems** when something went wrong for real customers.

1.3 What "end to end" means here

Stage	In simple words	Main question answered
1. Base-image creation	Build the template computer from scratch using a written recipe.	Is the template built the same way every time?
2. Validation	Test the template on every kind of machine before anyone uses it.	Does it boot, connect, and perform correctly everywhere?
3. Publishing	Put the approved template in the official catalog and copy it to every region.	Can teams and customers find and use the right version?
4. Staged rollout	Release it in small waves - a few servers first, then more.	If something is wrong, can we catch it while it is still small?
5. Production troubleshooting	Watch it in real use and fix issues quickly.	When something breaks, how fast can we find and fix it?

1.4 Why this matters

- **Every** virtual machine starts from an image. A bad image can break thousands of customer machines at once, so quality here protects the whole cloud.
- **Consistency:** the same recipe gives the same result every time, so problems are predictable and easy to reproduce.
- **Security:** images are the first line of defense - patched, hardened, and signed before anyone uses them.
- **Speed:** a good image pipeline turns a security fix into a safe, tested release in hours or days instead of weeks.
- **Scale:** "multiple environments and compute configurations" means the same image must work on Intel, AMD, Arm, GPU, virtual, and bare-metal machines across many regions.

2. Key Words in Plain English

Read this table once before the rest of the document. Every technical word used later is explained here.

Word	Simple meaning	Example
OCI	Oracle Cloud Infrastructure - Oracle's public cloud where companies rent computers, storage, and networks.	Like AWS or Azure, but run by Oracle.
Hypervisor	Software on a physical server that splits it into many isolated virtual machines.	KVM/QEMU-based hypervisor on an OCI host.
Host	The real, physical server that runs the hypervisor.	A rack server in an Oracle data center.
Guest / VM	A virtual machine running on top of a host.	A customer's Linux server with 4 CPUs.
Bare metal (BM)	Renting a whole physical server with no hypervisor sharing it.	BM.Standard.E5.192
OS image	A saved, ready-to-boot copy of a disk with an operating system already installed.	Oracle Linux 9 image, version 2026.09.01
Base image	The clean starting template, built from scratch, before any team-specific extras.	Minimal OS + drivers + agents.
Golden image	The approved, tested, official version everyone should use.	The image marked "production".
Custom image	An image created by a team or customer (not the default platform image).	An image imported from QCOW2.
Shape	OCI's word for a machine size and type (CPU type, count, memory, GPU).	VM.Standard.E5.Flex, VM.Standard.A1.Flex
Compute configuration	A combination of shape, CPU family, firmware, launch mode, and storage.	Arm VM with UEFI and paravirtualized disk.
OCID	Oracle Cloud ID - a unique ID for every resource in OCI.	ocid1.image.oc1..xxxx
Region	A geographic location with Oracle data centers.	us-ashburn-1, eu-frankfurt-1
Availability Domain (AD)	A separate data center inside a region.	AD-1, AD-2, AD-3
Fault Domain (FD)	A group of hardware inside an AD that fails independently.	FD-1, FD-2, FD-3
Object Storage	OCI's file-storage "warehouse" for large files.	A bucket holding image files.

Word	Simple meaning	Example
QCOW2 / .oci	File formats for saving disk images.	image-ol9.qcow2
Kernel	The core of the operating system that talks to hardware.	Linux UEK kernel
Driver	Small software that lets the OS use a piece of hardware.	virtio disk driver, NVIDIA GPU driver
Launch mode	How the VM sees its hardware: emulated (old style), paravirtualized (fast virtual devices), or native.	PARAVIRTUALIZED networking
BIOS / UEFI	The firmware that starts a computer before the OS loads. UEFI is the modern version.	UEFI_64 firmware
Secure Boot	Only lets trusted, signed software start during boot.	Shielded instance
cloud-init	A tool inside the image that sets up a new VM on first boot (hostname, SSH keys, disks).	Injects the customer's SSH key.
CVE	A publicly known security weakness with an ID number.	CVE-2024-XXXX
CI/CD	Automation that builds, tests, and releases software every time something changes.	A pipeline job.
Canary	A tiny first release to a few machines to detect problems early.	1% of hosts first.
Bake time	Waiting period after a rollout wave to watch for problems.	Wait 24 hours before the next wave.
Rollback	Going back to the last known good version.	Revert to image 2026.08.15.
Serial console	A text "emergency screen" to see a VM's boot messages even when the network is down.	Seeing a kernel panic message.

3. Architecture Diagram

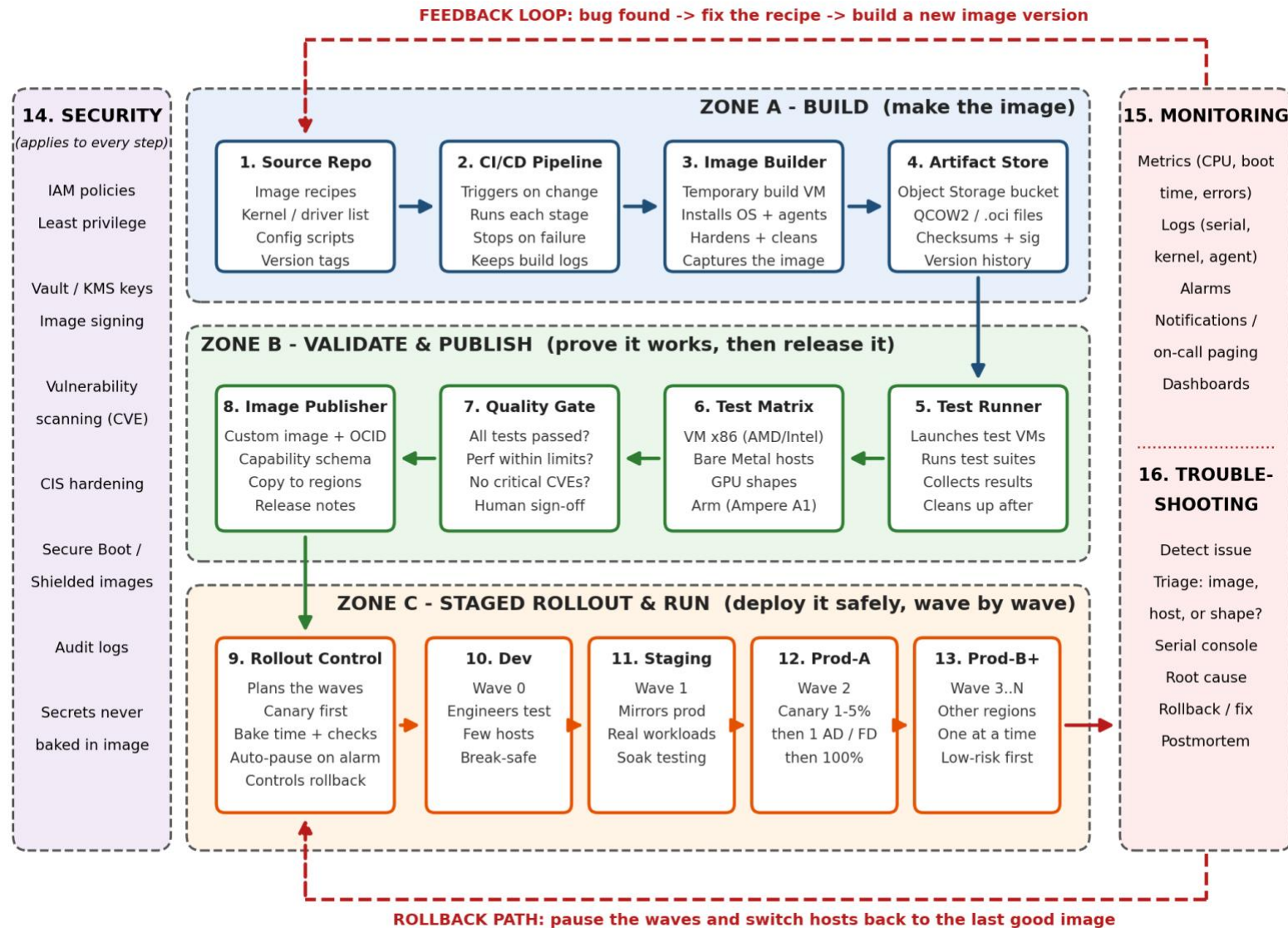


Figure 1 - End-to-end architecture. Numbers 1-16 match the component explanations in Sections 4 and 5.

4. The Architecture Diagram Explained, Piece by Piece

4.1 How to read the diagram

Visual element	What it means
Big dashed boxes (zones)	A stage of the life of an image. Zone A = build, Zone B = test and publish, Zone C = release and run.
Small solid boxes (1-16)	Individual components. The number matches the detailed explanation in Section 5.
Blue color	Build stage (Zone A) - making the image.
Green color	Validation and publishing stage (Zone B) - proving the image works and releasing it.
Orange color	Rollout stage (Zone C) - putting the image on real servers, wave by wave.
Purple side bar (14)	Security - it is drawn along the full height because it applies to every single step.
Red side bar (15, 16)	Monitoring and troubleshooting - watching production and fixing problems.
Solid arrows	The normal, forward path the image travels.
Red dashed arrows	Safety paths: the feedback loop (top) and the rollback path (bottom).
Snake shape (left-right, then right-left, then left-right)	Just a layout choice so arrows never cross. The image always moves forward: 1 to 13.

4.2 Zone A - Build (make the image)

This is the "kitchen" where the image is cooked from a written recipe. Nothing here is visible to customers.

- **Box 1 - Source Repo:** the written recipe (code and config files) for the image. Every change is reviewed and versioned.
- **Arrow 1 to 2:** a change to the recipe (for example a new kernel version) automatically starts the pipeline.
- **Box 2 - CI/CD Pipeline:** the "conductor" that runs every next step in order and stops the line if anything fails.
- **Arrow 2 to 3:** the pipeline asks the builder to create the image.
- **Box 3 - Image Builder:** starts a temporary computer, installs the OS and tools, hardens it, cleans it, and captures it as an image.

- **Arrow 3 to 4:** the finished image file is saved.
- **Box 4 - Artifact Store:** the warehouse (OCI Object Storage) that keeps every image version with its checksum and signature.

4.3 Zone B - Validate & Publish (prove it works, then release it)

This is the "inspection line". The image is tested hard before it is allowed out.

- **Arrow 4 to 5:** the new image is handed to the test system.
- **Box 5 - Test Runner:** launches test VMs from the image, runs all the test suites, collects results, and deletes the test machines afterwards.
- **Arrow 5 to 6:** tests are spread across every machine type.
- **Box 6 - Test Matrix:** the list of machine types the image must work on - Intel and AMD VMs, bare metal, GPU machines, and Arm machines.
- **Arrow 6 to 7:** all results are sent to the decision point.
- **Box 7 - Quality Gate:** a pass/fail checkpoint. It only opens if all tests pass, performance is normal, and there are no serious security issues. A human also signs off.
- **Arrow 7 to 8:** only a passing image moves on.
- **Box 8 - Image Publisher:** registers the image in OCI (it gets an OCID), attaches its capability settings, copies it to every region, and writes release notes.

4.4 Zone C - Staged Rollout & Run (deploy it safely, wave by wave)

This is the "delivery" stage. The image reaches real servers in small, controlled steps.

- **Arrow 8 to 9:** the published image is handed to the rollout system.
- **Box 9 - Rollout Control:** plans the waves, starts with a canary, waits (bake time), checks health, and pauses or rolls back automatically if alarms fire.
- **Box 10 - Dev (Wave 0):** engineers' own environment. Safe to break.
- **Box 11 - Staging (Wave 1):** a copy of production with realistic workloads for longer "soak" testing.
- **Box 12 - Prod-A (Wave 2):** the first production region. Starts at 1-5% of hosts (canary), then one availability domain or fault domain, then 100%.
- **Box 13 - Prod-B+ (Waves 3..N):** all other regions, one at a time, low-risk regions first.
- **Orange arrows between 10-13:** the image only moves to the next wave after the previous wave is healthy for its full bake time.

4.5 The side bars

- **Box 14 - Security (purple, left):** access control, encryption keys and image signing, vulnerability scanning, CIS hardening, Secure Boot, audit logs, and the rule "no passwords or secrets inside an image". It touches every zone.

- **Box 15 - Monitoring (red, right, top):** metrics, logs, alarms, paging, and dashboards that watch every environment.
- **Box 16 - Troubleshooting (red, right, bottom):** the process used when something goes wrong: detect, triage, investigate, fix or roll back, and write a postmortem.
- **Red arrow from Prod-B+ to Monitoring:** health signals from all running servers flow into monitoring.

4.6 The two safety loops (red dashed arrows)

Arrow	Path	What it means in simple words
Feedback loop (top)	Monitoring / Troubleshooting to Box 1 Source Repo	When a bug is found, the fix is written into the recipe, and a brand-new image version is built. We never "hand-patch" a live image.
Rollback path (bottom)	Troubleshooting to Box 9 Rollout Control	If a new image causes problems, the rollout is paused and servers are switched back to the last known good image.

5. Every Component Explained in Detail

Each component below has four parts: **In simple terms**, **Detailed breakdown**, **Why it matters**, and an **Everyday analogy**. The numbers match the diagram.

5.1 Component 1 - Source Repository (the recipe book)

In simple terms

A Git repository that stores the written instructions for how to build the image. If it is not written here, it does not go into the image.

Detailed breakdown

- **Image templates:** files (for example Packer templates) that describe the starting point, the machine to build on, and the steps to run.
- **OS install answers:** kickstart or similar files that answer install questions automatically (disk layout, language, packages).
- **Package list:** the exact list of software and versions to install, pinned so builds are repeatable.
- **Kernel and driver versions:** which kernel, which virtio drivers, which GPU / network drivers, per shape family.
- **Configuration scripts:** Ansible, shell, or cloud-init snippets that set time sync, logging, SSH settings, and agents such as Oracle Cloud Agent.
- **Hypervisor settings:** host-level configuration, such as KVM/QEMU versions, CPU features exposed to guests, and host kernel parameters.
- **Test definitions:** the tests that the image must pass, stored next to the recipe.
- **Version tags and changelog:** every release gets a tag such as ol9-2026.09.01 so anyone can rebuild the exact same image later.
- **Code review:** every change needs a pull request and approval by another engineer.

Why it matters

Makes images repeatable, reviewable, and auditable. If production breaks, we can see exactly what changed and when.

Everyday analogy: A cookbook. Every chef follows the same written recipe, so every cake comes out the same.

5.2 Component 2 - CI/CD Pipeline (the conductor)

In simple terms

Automation that notices a change in the recipe and runs every step - build, store, test, publish - in the right order, without manual clicking.

Detailed breakdown

- **Triggers:** a merged code change, a scheduled rebuild (for example weekly security patches), or a manual start for emergencies.
- **Stages:** lint and check the recipe, build, upload, scan, test, gate, publish, start rollout.
- **Fail-fast rule:** if any stage fails, the pipeline stops and nothing moves forward.
- **Parallel jobs:** builds for x86 and Arm, or tests for different shapes, run at the same time to save hours.
- **Logs and artifacts:** every run keeps logs, test reports, and the exact image version it produced.
- **Credentials:** the pipeline uses its own limited OCI identity (for example an instance principal or API key stored in a vault), never a person's password.
- **Notifications:** posts success or failure to chat or email so the team knows immediately.

Why it matters

Removes human error and makes every release follow the same safe path.

Everyday analogy: A factory assembly line that moves the product from station to station and stops the belt if a station reports a defect.

5.3 Component 3 - Image Builder (the kitchen)

In simple terms

The machine that actually cooks the image: it starts a temporary computer, installs everything, cleans it, and saves it as an image.

Detailed breakdown

- **Start from a known base:** an official Oracle Linux (or other OS) platform image or an ISO install.
- **Temporary build VM:** launched in an isolated build compartment and subnet on OCI.
- **Install and update:** apply the latest OS packages and security patches.
- **Install kernel and drivers:** virtio drivers for virtual disks and network, NVMe support, NVIDIA drivers for GPU images, Mellanox/RDMA drivers for high-performance networking where needed.
- **Install agents:** Oracle Cloud Agent, monitoring and logging agents, OS management agent, cloud-init.
- **Configure boot:** GRUB settings, serial console output (so boot messages can be read later), UEFI and Secure Boot support, correct initramfs.
- **Harden:** apply CIS benchmark settings, disable unused services, set firewall defaults, SSH key-only login.
- **Generalize and clean:** remove SSH host keys, machine-id, logs, bash history, temporary users, and cloud-init state so every new VM starts fresh and unique.

- **Capture:** stop the VM and create a custom image from its boot volume; then delete the temporary VM.
- **Hypervisor builds:** in the same way, the host-side hypervisor packages (for example KVM/QEMU and host kernel) are built as versioned packages or host images.

Why it matters

A clean, generalized image means every VM created from it is identical and has no leftover secrets.

Everyday analogy: A kitchen that bakes a cake from the recipe, decorates it, then wipes the counter clean so the next cake starts fresh.

5.4 Component 4 - Artifact Store (the warehouse)

In simple terms

Safe storage for every image version, usually an OCI Object Storage bucket.

Detailed breakdown

- **Export:** the custom image is exported to Object Storage as a file (QCOW2 or the OCI .oci format that includes metadata).
- **Naming:** clear names such as ol9-uek-x86_64-2026.09.01.oci.
- **Checksum:** a SHA-256 fingerprint is stored so anyone can prove the file was not changed.
- **Signature:** the checksum is signed with a key from OCI Vault so the image can be trusted.
- **Versioning and retention:** old versions are kept for a set time so rollback is always possible.
- **Access control:** only the pipeline can write; only approved groups can read. Pre-authenticated requests (time-limited links) can share a file safely.
- **Metadata file:** a small manifest listing kernel version, package list, build time, and git commit.

Why it matters

Nothing is ever lost, and the exact bits that were tested are the exact bits that ship.

Everyday analogy: A locked warehouse where every box is labelled, sealed, and logged.

5.5 Component 5 - Test Runner (the inspector)

In simple terms

Automation that launches test machines from the new image, runs many tests, and reports pass or fail.

Detailed breakdown

- **Launch:** creates test instances from the new image on each shape in the test matrix.

- **Boot tests:** does it boot? how long does it take? are there errors in the serial console log?
- **First-boot tests:** did cloud-init run, set the hostname, add SSH keys, and grow the disk?
- **Network tests:** gets an IP, reaches the internet and OCI services, correct MTU (for example 9000 jumbo frames), DNS works, secondary VNICs attach.
- **Storage tests:** boot volume and block volumes attach (iSCSI and paravirtualized), NVMe local disks are detected, resize works, read/write speed is normal.
- **Hypervisor functional tests:** start, stop, reboot, live migration, CPU and memory hot-plug where supported, correct CPU features visible to the guest.
- **Performance tests:** CPU, memory, disk, and network benchmarks compared to the previous image.
- **Security tests:** vulnerability scan, CIS check, Secure Boot verified, no default passwords, no open unexpected ports.
- **Agent tests:** Oracle Cloud Agent, monitoring, and logging agents report in.
- **Upgrade tests:** a VM from the old image can be patched up to match the new one.
- **Cleanup:** deletes every test resource so no cost or leftovers remain.

Why it matters

Finds problems in a lab instead of on customer machines.

Everyday analogy: A car inspector who test-drives every new model on highways, hills, rain, and snow before it goes to the showroom.

5.6 Component 6 - Test Matrix (every kind of machine)

In simple terms

The list of machine types ("compute configurations") that every image must be tested on.

Detailed breakdown

- **CPU families:** Intel x86, AMD x86, and Arm (Ampere A1 / A2). Arm needs its own image build because the instructions are different.
- **VM shapes:** flexible VMs such as VM.Standard.E5.Flex and VM.Standard3.Flex, tested at small and large sizes.
- **Bare metal shapes:** whole physical servers, where the image runs without the hypervisor and must have real hardware drivers.
- **GPU shapes:** machines with NVIDIA GPUs, needing correct GPU driver and CUDA versions.
- **Dense I/O shapes:** machines with local NVMe disks.
- **Firmware:** BIOS and UEFI boot.
- **Launch modes:** paravirtualized, emulated, and native.
- **Security modes:** normal and Shielded (Secure Boot, Measured Boot).

- **Regions:** at least one test in a commercial region and, where relevant, special regions (for example government regions).

Why it matters

An image that works on one machine may fail on another. Testing every combination prevents surprises.

Everyday analogy: Testing a new shoe in every size, not just size 9.

5.7 Component 7 - Quality Gate (the checkpoint)

In simple terms

A strict yes/no decision point. The image moves forward only if every rule is met.

Detailed breakdown

- **Automatic rules:** 100% of required tests pass; boot time and performance are within an allowed range (for example no more than 5% slower); zero critical or high security findings.
- **Change review:** a person reads the changelog and test report and approves.
- **Risk rating:** small changes (a package update) can move faster; big changes (a new kernel or hypervisor version) need longer bake times.
- **Recorded decision:** who approved, when, and why - kept for audits.

Why it matters

Stops a bad image from ever reaching customers.

Everyday analogy: Airport security. No boarding pass without passing the scanner.

5.8 Component 8 - Image Publisher (the catalog)

In simple terms

Makes the approved image officially available in OCI, in every region that needs it.

Detailed breakdown

- **Import / register:** creates the custom image in OCI from the stored file; OCI gives it an OCID.
- **Capability schema:** attaches the Compute Image Capability Schema that tells OCI which firmware, launch modes, and features the image supports.
- **Shape compatibility:** lists which shapes are allowed to use the image so customers cannot pick an unsupported combination.
- **Cross-region copy:** exports and imports (or copies) the image to every target region.
- **Sharing:** shares with the right tenancies or compartments, or lists it in the internal catalog or OCI Marketplace.
- **Tags:** free-form and defined tags such as version, os, arch, status=candidate or status=production.

- **Release notes:** what changed, what was fixed, known issues.
- **Deprecation:** older images are marked deprecated and later removed on a published schedule.

Why it matters

Teams and customers always get the right, approved version, in their own region.

Everyday analogy: A publisher putting a new book edition on every bookstore shelf and marking the old edition as outdated.

5.9 Component 9 - Rollout Controller (the traffic light)

In simple terms

Controls how fast the new image or hypervisor version reaches real servers, and stops automatically if something looks wrong.

Detailed breakdown

- **Wave plan:** Dev, then Staging, then the first production region, then the rest of the regions.
- **Blast-radius control:** inside a region, go canary (1-5% of hosts), then one fault domain, then one availability domain, then all.
- **Bake time:** wait a fixed time after each wave (for example hours in dev, a day or more in production) while watching metrics.
- **Health checks:** boot success rate, VM launch success rate, kernel crash count, host error rate, customer tickets.
- **Automatic pause:** if an alarm fires, the rollout stops by itself.
- **Rollback:** points the default back to the last good image, and for hosts, reverts the hypervisor package or reboots into the previous version.
- **Change windows and freezes:** no rollouts during holidays or major events.
- **How the switch happens:** for guest images, by updating the default image OCID in instance configurations or launch templates; for hypervisor hosts, by draining VMs (live migration), updating the host, rebooting, and returning it to service.

Why it matters

Keeps any problem small. A bug on 1% of servers is an incident; on 100% it is an outage.

Everyday analogy: Opening a new road lane by lane, watching traffic after each lane opens.

5.10 Component 10 - Dev Environment (Wave 0)

In simple terms

The engineers' playground. The first real place the image runs.

Detailed breakdown

- **Small number of hosts** and test tenancies.
- **Break-safe:** no real customers, so risky tests are allowed.
- **Fast feedback:** bake time is short (hours).
- **Used for:** debugging new kernels, trying new hypervisor features, reproducing production bugs.

Why it matters

Catches obvious problems at zero customer risk.

Everyday analogy: A practice kitchen.

5.11 Component 11 - Staging Environment (Wave 1)

In simple terms

A near-copy of production used for longer, realistic testing.

Detailed breakdown

- **Same shapes, same network design, same configuration** as production, just smaller.
- **Real-like workloads:** internal services and synthetic load run here.
- **Soak testing:** the image runs for days to find slow problems such as memory leaks or clock drift.
- **Upgrade rehearsal:** the exact production rollout steps are practiced here first.

Why it matters

Finds problems that only appear over time or under real load.

Everyday analogy: A dress rehearsal before opening night.

5.12 Component 12 - Production Region A (Wave 2)

In simple terms

The first real customer region to get the new version.

Detailed breakdown

- **Chosen on purpose:** usually a region with good monitoring and moderate traffic.
- **Canary first:** 1-5% of hosts, then one fault domain, then one availability domain, then the whole region.
- **Longer bake time** and close watching by the on-call engineer.
- **Customer-facing signals** (support tickets, launch failures) are watched in addition to system metrics.

Why it matters

The first time real customer workloads meet the new version, so it is watched most carefully.

Everyday analogy: A soft launch of a new store in one city.

5.13 Component 13 - Production Regions B and beyond (Waves 3..N)

In simple terms

Every other region, updated one after another.

Detailed breakdown

- **One region (or small group) at a time** so two regions are never broken together.
- **Low-risk first:** smaller or less critical regions go before the largest ones.
- **Special regions** (government, sovereign, dedicated) may follow their own approval rules.
- **Completion:** when all regions are done, the image is marked production and the older version starts its deprecation countdown.

Why it matters

Keeps customers in other regions safe while confidence grows.

Everyday analogy: Opening the rest of the store chain city by city after the first store succeeds.

5.14 Component 14 - Security (the guard at every door)

In simple terms

Rules and tools that keep the images and hypervisor safe at every step.

Detailed breakdown

- **IAM policies:** only the pipeline and specific groups can create, change, or publish images (least privilege).
- **Compartments:** build, test, and production resources are separated.
- **OCI Vault / KMS:** stores signing keys and secrets; encrypts storage.
- **Image signing:** proves the image came from our pipeline and was not changed.
- **Vulnerability scanning:** checks every package against known CVEs before and after release.
- **CIS hardening:** follows an industry security checklist.
- **Secure Boot / Shielded instances:** only trusted, signed boot software can start.
- **No secrets in images:** no passwords, keys, or tokens are ever baked in; they are injected at launch.
- **Audit logs:** OCI Audit records who did what and when.
- **Hypervisor isolation:** keeps one customer's VM from seeing another's memory or data; hypervisor patches for CPU-level vulnerabilities are high priority.

Why it matters

One insecure image would be copied to thousands of machines, so security must be built in.

Everyday analogy: A bank vault with guards, cameras, and a visitor log at every door.

5.15 Component 15 - Monitoring (the eyes and ears)

In simple terms

Tools that constantly watch every server and every image version and raise an alarm when something looks wrong.

Detailed breakdown

- **Metrics:** CPU, memory, disk, network, boot time, launch success rate, host error counts (OCI Monitoring).
- **Logs:** serial console output, kernel logs, cloud-init logs, agent logs, hypervisor logs (OCI Logging).
- **Alarms:** thresholds such as "launch failure rate above 1%" or "kernel panic count above 0".
- **Notifications:** alarms go to email, chat, or on-call paging through OCI Notifications.
- **Dashboards:** one screen per image version and per region comparing new vs. old.
- **Version labels:** every metric is tagged with the image and hypervisor version, so a problem can be tied to a release immediately.

Why it matters

You cannot fix what you cannot see. Good monitoring turns a customer outage into an early warning.

Everyday analogy: A car dashboard with warning lights.

5.16 Component 16 - Troubleshooting (the doctor)

In simple terms

The process and tools used to find and fix problems in production quickly.

Detailed breakdown

- **Detect:** an alarm, dashboard trend, or customer ticket.
- **Triage:** decide severity and ask the key question: is it the image, the host/hypervisor, the shape, or the customer's own setup?
- **Contain:** pause the rollout; if serious, roll back.
- **Investigate:** serial console, kernel logs, crash dumps, comparing new vs. old version, reproducing in dev on the same shape.
- **Fix:** write the fix into the recipe (Component 1), build a new version, and send it through the full pipeline.
- **Postmortem:** a blameless write-up: what happened, why, and what test is added so it never happens again.

Why it matters

Keeps outages short and makes the system stronger after every incident.

Everyday analogy: A doctor who diagnoses, treats, and then updates the health checklist.

6. The Complete Workflow, Step by Step

This section follows one image version from the first idea to running safely in every region. The same flow is used for hypervisor updates, with small differences noted at the end.

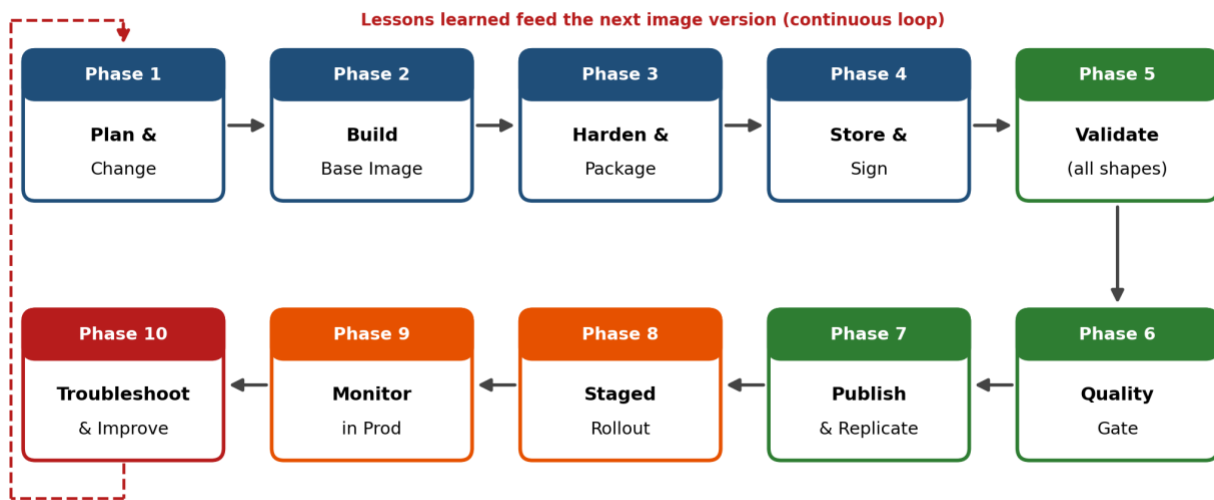


Figure 2 - The 10 phases of the workflow. The red dashed line shows the continuous improvement loop.

Phase 1 - Plan the change

Goal: Decide what the new version needs and why.

Step	What happens (detail)	In simple words
1.1	A need appears: a security patch (CVE), a new kernel, a new shape to support, a bug fix, or a customer request.	Something needs to change.
1.2	The engineer writes a short plan: what changes, which shapes are affected, the risk level.	Think before you build.
1.3	The recipe is updated in the Source Repo on a new branch.	Edit the cookbook.
1.4	A pull request is opened and reviewed by another engineer.	A second pair of eyes.
1.5	After approval, the change is merged. This triggers the pipeline.	Press "start" on the factory.

Phase 2 - Build the base image

Goal: Turn the recipe into a real image.

Step	What happens (detail)	In simple words
2.1	The CI/CD pipeline starts and checks the recipe for mistakes (linting, syntax).	Proof-read the recipe.

Step	What happens (detail)	In simple words
2.2	The builder launches a temporary build VM from the approved starting image in an isolated build network.	Get a clean workbench.
2.3	The OS is updated and the chosen kernel is installed.	Lay the foundation.
2.4	Drivers are installed per shape family (virtio, NVMe, GPU, RDMA) and x86 and Arm are built in parallel.	Add the right parts for each machine type.
2.5	Agents and cloud-init are installed and configured.	Add the tools every VM needs.
2.6	Boot settings are set: GRUB, serial console output, UEFI / Secure Boot support.	Make sure it can start and show its messages.

Phase 3 - Harden and package

Goal: Make it secure and generic.

Step	What happens (detail)	In simple words
3.1	Apply CIS hardening, firewall defaults, and key-only SSH.	Lock the doors and windows.
3.2	Remove unique data: SSH host keys, machine-id, logs, history, temporary users.	Wipe fingerprints so every copy is fresh.
3.3	Shut down the build VM and capture the boot volume as a custom image.	Take the snapshot.
3.4	Delete the temporary build VM.	Clean up.

Phase 4 - Store and sign

Goal: Save the exact bits safely.

Step	What happens (detail)	In simple words
4.1	Export the image to the Object Storage bucket (QCOW2 or .oci).	Put it in the warehouse.
4.2	Calculate a SHA-256 checksum and sign it with a key from OCI Vault.	Seal the box with a tamper-proof sticker.
4.3	Save a manifest: version, kernel, packages, git commit, build time.	Attach a packing list.
4.4	Run a first vulnerability scan on the package list.	Quick security check before testing.

Phase 5 - Validate on every configuration

Goal: Prove it works everywhere.

Step	What happens (detail)	In simple words
5.1	The Test Runner launches test VMs and bare-metal instances across the full test matrix.	Try it on every kind of machine.
5.2	Run boot and first-boot tests (boot time, serial log, cloud-init).	Does it start and set itself up?
5.3	Run network and storage tests (IP, DNS, MTU, volume attach, NVMe, speed).	Can it talk and save data?
5.4	Run hypervisor tests (reboot, stop/start, live migration, CPU features).	Does it behave well on the host?
5.5	Run performance benchmarks and compare with the previous version.	Is it as fast as before?
5.6	Run security and compliance scans.	Is it still safe?
5.7	Collect all results into one report and delete test resources.	Write the report card and tidy up.

Phase 6 - Quality gate

Goal: Decide: go or no-go.

Step	What happens (detail)	In simple words
6.1	Automatic check: all tests passed, performance within limits, no critical CVEs.	The machine checks the report card.
6.2	Human review of the report and changelog; approval recorded.	A person signs off.
6.3	If it fails: the pipeline stops, the team is notified, and the fix goes back to Phase 1.	Back to the kitchen.

Phase 7 - Publish and replicate

Goal: Make it available everywhere.

Step	What happens (detail)	In simple words
7.1	Register the image in OCI and attach the capability schema and shape compatibility.	Put it in the catalog with the right labels.
7.2	Copy the image to every target region.	Stock every store.

Step	What happens (detail)	In simple words
7.3	Tag it as candidate and share it with the right compartments or tenancies.	Mark it "new, not yet default".
7.4	Publish release notes.	Tell people what changed.

Phase 8 - Staged rollout

Goal: Release it slowly and safely.

Step	What happens (detail)	In simple words
8.1	Wave 0 - Dev: make it the default; short bake time; watch metrics.	Try it in the practice kitchen.
8.2	Wave 1 - Staging: realistic workloads; multi-day soak.	Dress rehearsal.
8.3	Wave 2 - First production region: canary 1-5% of hosts, then one fault domain, then one availability domain, then all.	Soft launch in one city, street by street.
8.4	Waves 3..N - other regions, one at a time, low-risk first.	Open the other cities one by one.
8.5	After each wave, the controller checks health; an alarm pauses the rollout automatically.	A traffic light between each step.
8.6	When all regions are healthy, tag the image as production and start deprecating the old one.	Make it the official version.

Phase 9 - Monitor in production

Goal: Keep watching.

Step	What happens (detail)	In simple words
9.1	Dashboards compare new vs. old version by region and shape.	Side-by-side health check.
9.2	Alarms watch launch failures, boot time, kernel crashes, host errors.	Warning lights.
9.3	Support tickets are tagged with the image version.	Listen to customers.

Phase 10 - Troubleshoot and improve

Goal: Fix problems and learn from them.

Step	What happens (detail)	In simple words
10.1	An alarm or ticket starts an incident; on-call engineer responds.	Someone picks up the phone.
10.2	Triage: image, hypervisor/host, shape, or customer configuration?	Find which part is sick.
10.3	Contain: pause the rollout or roll back to the last good version.	Stop the bleeding.
10.4	Investigate with serial console, logs, crash dumps, and reproduction in dev.	Run the diagnosis.
10.5	Fix in the recipe and send a new version through Phases 1-8.	Cure it properly, not with tape.
10.6	Write a postmortem and add a new test so it cannot happen again.	Update the checklist.

How the workflow differs for hypervisor updates

The hypervisor lives on the physical host, not inside the customer's VM, so it cannot be "re-launched" like an image. The same phases apply, with these differences:

- **Build:** the hypervisor (for example KVM/QEMU and the host kernel) is built as versioned packages or a host OS image.
- **Validate:** tests focus on VM launch, isolation, live migration, CPU feature exposure, performance, and compatibility with every guest image and shape.
- **Rollout:** each host is drained first - running VMs are live-migrated to other hosts (or customers are given a maintenance window) - then the host is updated, rebooted, health-checked, and returned to service.
- **Rollback:** the host reboots into the previous hypervisor version, which is kept installed until the new one is proven.
- **Pace:** usually slower, because a host problem affects every VM on that host.

7. Multiple Environments and Compute Configurations

7.1 Environments

Environment	Purpose	Who uses it	Risk if broken	Typical bake time
Dev	Build and try new versions quickly.	Image and hypervisor engineers	Very low	Hours
Test / QA	Run the automated test matrix.	Pipeline (automation)	Very low	Duration of test run
Staging	Production copy for soak and rehearsal.	Internal teams, synthetic load	Low	Several days
Prod - first region	First real customers, canary first.	Customers	Medium (small blast radius)	One day or more per step
Prod - other regions	Everyone else, one region at a time.	Customers	High if uncontrolled	One day or more per region

7.2 Compute configurations (the test matrix)

Configuration	Example	What is special	What we check
Intel x86 VM	VM.Standard3.Flex	Most common VM type	Boot, network, storage, CPU features
AMD x86 VM	VM.Standard.E5.Flex	Different CPU features than Intel	CPU flags, performance, live migration
Arm VM	VM.Standard.A1.Flex	Different CPU architecture; needs its own build	Arm kernel, drivers, package availability
Bare metal	BM.Standard.E5.192	No hypervisor; real hardware	Hardware drivers, NIC, iSCSI boot volume
GPU	VM.GPU.A10.1 / BM.GPU.H100.8	NVIDIA GPUs	GPU driver, CUDA, device detection
Dense I/O	BM.DenseIO shapes	Local NVMe disks	NVMe detection and speed
Firmware	BIOS vs. UEFI_64	Different boot path	Bootloader works in both
Launch mode	Paravirtualized / Emulated / Native	Different virtual device types	Disk and network drivers for each
Shielded	Secure Boot + Measured Boot	Only signed boot code runs	Signed kernel and bootloader

Simple takeaway: one image version is not "done" until it works on every row of this table, in every environment, in the right order.

8. Production Troubleshooting Playbook

These are the kinds of real problems this role handles, explained simply.

Symptom	Likely cause	How to check	Fix
New VMs do not boot	Missing driver in initramfs, wrong bootloader for UEFI/BIOS, bad kernel	Serial console log on the failing shape	Roll back; rebuild with correct driver or bootloader
VM boots but cannot be reached	cloud-init failed, network config wrong, firewall too strict	Serial console, cloud-init logs	Fix network or cloud-init config in the recipe
Slow network or dropped packets	Wrong MTU, driver version, offload settings	Compare with old image; ping and iperf tests	Correct MTU or driver; add a test
Block volume will not attach	iSCSI or paravirtualized setting missing	Check attach type and guest logs	Include the right tools/drivers; test both attach types
Works on Intel, fails on Arm or AMD	Package or driver missing for that architecture or CPU flag	Re-run tests on that shape only	Fix the per-architecture recipe
GPU not detected	NVIDIA driver and kernel mismatch	nvidia-smi, kernel logs	Pin compatible driver and kernel versions
Random kernel crashes	Kernel or hypervisor bug	Crash dump, host logs, version comparison	Roll back; patch; add stress test
Performance slower than before	New kernel settings, CPU mitigations, hypervisor change	Benchmarks old vs. new on same shape	Tune or revert the change
Live migration fails	Host versions or CPU features differ	Hypervisor logs on source and target	Align CPU feature sets; fix hypervisor
Only one region has errors	Image copy incomplete, regional config difference	Compare image checksum and region settings	Re-copy image; fix regional setting

The general troubleshooting method

- **1. Stop the spread:** pause the rollout first, investigate second.
- **2. Narrow it down:** which version, which shape, which region, which host? Compare healthy vs. unhealthy.
- **3. Look at the boot story:** the serial console shows exactly where startup failed.

- **4. Compare old vs. new:** the diff between versions usually points to the cause.
- **5. Reproduce:** make the same failure happen in dev on the same shape.
- **6. Fix in the recipe, not by hand:** so the fix is permanent and repeatable.
- **7. Add a test:** every incident becomes a new automatic check.

9. Safety Nets, Success Measures, and a 60-Second Explanation

9.1 Safety nets built into the system

- **Fail-fast pipeline:** any failed step stops everything.
- **Quality gate:** no image moves forward without passing every rule.
- **Canary and waves:** problems stay small.
- **Bake time:** time to notice slow problems.
- **Automatic pause:** alarms stop the rollout without waiting for a human.
- **Keep the last good version:** rollback is always one step away.
- **Signed images:** only trusted images can be used.
- **Postmortems:** every incident makes the system stronger.

9.2 How success is measured

Measure	What it tells you	Good direction
Build success rate	How reliable the build pipeline is	Up
Time from fix to production	How fast a security patch reaches customers	Down
Test coverage across shapes	How many configurations are checked	Up
VM launch success rate	Whether new VMs start correctly	Up (close to 100%)
Boot time	How quickly a VM is ready	Down
Rollbacks per release	How often bad versions got through	Down
Incidents caught in canary vs. full rollout	Whether the waves are doing their job	More caught early
Mean time to recover (MTTR)	How fast problems are fixed	Down
Open critical CVEs in current images	Security health	Zero

9.3 The 60-second explanation (for interviews)

"On Oracle Cloud, every virtual machine starts from an operating-system image and runs on a hypervisor. I owned that whole layer. I built images from version-controlled recipes through an automated pipeline, stored and signed them, and tested each version across all our compute configurations - Intel, AMD, Arm, GPU, and bare metal - covering boot, networking, storage, performance, and security. Only images that passed a strict quality

gate were published and copied to every region. Then I rolled them out in waves - dev, staging, a canary in the first production region, then region by region - with bake times and automatic pause on alarms. When issues came up in production, I led troubleshooting using serial console logs, crash dumps, and version comparisons, rolled back when needed, fixed the root cause in the recipe, and added a test so it would not happen again."